Claude-powered paper audit pipeline — design report

Repo: `sednabcn/LLM-HypatiX-DEV`
Scope: audits `**/*_patched.tex` against `hypatiax/data/results/`, using the Claude API, independent of the existing `ci_paper_audit.yml` gate.

---

1 Why this exists

`ci_paper_audit.yml` already does deterministic, rule-based auditing of the paper against results data. This pipeline adds a second, LLM-driven layer on top, for two jobs a deterministic script can't do:

1. **Numeric consolidation** — when a paper claim's underlying results have multiple files (reruns, seeds, shards), decide the single authoritative value and *why*, then propose (and in narrow cases, auto-commit) the fix.
2. **The narrative "open/flagging points" report** — a human-readable account of what's stale, what's missing, and what's actively blocked, on top of the raw findings JSON.

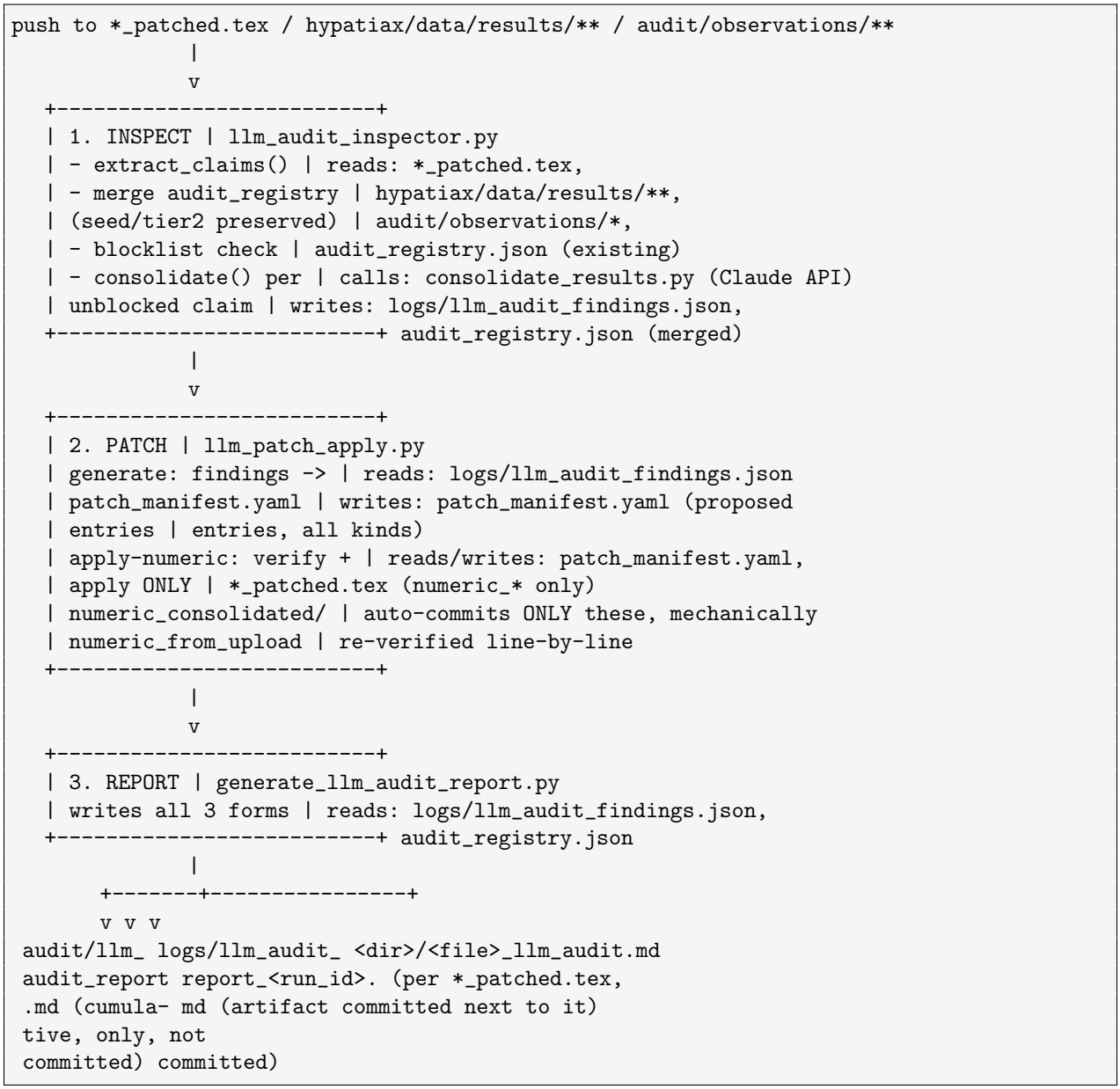
It deliberately does **not** try to replicate the deep, cascading, multi-file investigations run manually (the `68_74-issue.txt` / `leak_report.tex` style sessions). Those stay human+Claude, outside CI. This pipeline's only relationship to that work is: **it reads the block list those sessions produce, and it never overwrites a number in a table/figure that list says is currently invalid.**

2 Two-tier design

	Tier 1 — automated, in CI	**Tier 2 — human+Claude, outside CI**
What it is	`llm_audit_inspector.py`, one Claude call per claim, bounded	The kind of session in `68_74-issue.txt` / `notes-nguyen12.txt` — iterative, requests more files, hypotheses get overturned, scope legitimately expands
Trigger	push to `*_patched.tex` / `hypatiax/data/results/**` / `audit/observations/**`, or manual dispatch	you, manually, whenever
Output	`logs/llm_audit_findings.json`, updates to `audit_registry.json`, three forms of report	Incident docs like `leak_report.tex`, filed back in via `audit/observations/*.json` (`type:"block"`)

	Tier 1 — automated, in CI	**Tier 2 — human+Claude, outside CI**
Can it auto-commit?	Yes, but **only** a single-digit numeric substitution, mechanically re-verified (no LLM trust)	Never automatically — always a human-reviewed patch
Relationship	**Reads** Tier 2's block list before doing anything; refuses to touch any claim inside a blocked table/figure	Produces the block list; the only way to remove a block is editing `audit_registry.json` directly

3 Data flow



Separately, whenever a Tier-2 investigation produces a block:

```
you file audit/observations/<name>.json (type: "block")
              |
              v
next Tier-1 run's INSPECT step merges it into audit_registry.json
as a tier2_upload entry -> its 'blocks' list is now enforced
```

4 File inventory — what goes where in the repo

All paths below are relative to the repo root.

#	File	Repo destination	Purpose
1	`ci_llm_paper_audit.yml`	`.github/workflows/`	The workflow itself: 3 jobs (`inspect` → `patch` → `report`), independent of `ci_paper_audit.yml`
2	`llm_audit_inspector.py`	`scripts/patches/`	Tier-1 entry point. Extracts numeric claims from `*_patched.tex`, tracks table/figure labels, checks the blocklist, calls `consolidate_results.py` for unblocked claims, merges results into `audit_registry.json`
3	`consolidate_results.py`	`scripts/patches/`	The **only** file that calls the Claude API. One bounded call per claim: given a section's result summary files, returns the authoritative value + method + confidence + provenance as strict JSON
4	`llm_patch_apply.py`	`scripts/patches/`	Deterministic, no Claude calls. `generate` writes findings into `patch_manifest.yaml` (existing schema); `apply-numeric` mechanically verifies and applies *only* numeric single-token substitutions
5	`generate_llm_audit_report.py`	`scripts/patches/`	Writes the open/flagging-points report in all three requested forms (cumulative, timestamped, per-tex-file)
6	`audit_registry_lib.py`	`scripts/patches/`	Shared library: load/save `audit_registry.json`, compute the blocklist, merge Tier-1 findings without ever touching seed/`tier2_upload` entries

#	File	Repo destination	Purpose
7	seed_audit_registry.py	scripts/patches/	One-time (idempotent) importer — merges a seed file into audit_registry.json
8	plan_action_seed.json	audit/seed/	The actual 13 PLAN-ACTION.txt items + item 14 (the ground-truth leak from leak_report.tex), pre-converted to the registry schema
9	README.md (observations)	audit/observations/	Documents both upload paths (committed .md/.json, and workflow_dispatch ad-hoc) and the type:"block" schema for filing Tier-2 findings

Files this pipeline generates at runtime (not delivered — created by the workflow)

File	Path	Committed?	Notes
audit_registry.json	repo root	Yes	merged and committed each run
patch_manifest.yaml	repo root	Yes	existing file; this pipeline only appends
logs/llm_audit_findings.json	logs/	No	artifact only
audit/llm_audit_report.md	audit/	Yes	cumulative, overwritten each run
logs/llm_audit_report_<run_id>.md	logs/	No	artifact only
<dir>/<texfile>_llm_audit.md	next to each *_patched.tex	Yes	one per patched tex file
audit/incident_reports/*.tex	audit/incident_reports/	manual	you commit these; pipeline only reads/links

5 Setup steps (in order)

1. Copy files 1–9 from Section 4 into the listed paths.
2. pipinstallanthropicpyyaml (already in the workflow's install step — nothing to do if only running in CI).
3. Confirm secrets.ANTHROPIC_API_KEY is set at the repo level (it already is, per ci_paper_audit.yml).
4. **Seed the registry once:**

```
python3 scripts/patches/seed_audit_registry.py \
```

```
  --seed audit/seed/plan_action_seed.json \
  --registry audit_registry.json
git add audit_registry.json && git commit -m \
  "chore: seed audit_registry.json from PLAN-ACTION.txt"
```

This immediately puts Tables 9–13, 15, `tab:hybrid_all`, `tab:runtime`, `tab:timing`, and Figures 9–13 on the blocklist, and items #4, #10, #12, #13 in as `resolved` so Tier 1 doesn't re-flag them.

5. If `audit/incident_reports/leak_report.tex` already exists, commit it now too — the seed entry for item 14 already points `linked_report` at that exact path.

6. **Fix the one assumption made blind:** `resolve_section_to_results()` in `llm_audit_inspector.py` assumes a `paper_targets.json` schema of the form `{"sections":[{"section":...,"experiment_ids":[...]}]}`, which was invented for lack of the real file. Supply the real schema (or the equivalent lookup already inside `hypatia_inspector.py`) to rewire that one function — everything else is independent of it.

6 Known limitations (found via testing, not theoretical)

- **Label detection is regex-based, not a real tex parser.** `LABEL_RE` catches `\label{tab:...}`/`\label{fig:...}`; `PROSE_LABEL_RE` catches literal "Table N" / "Figure N". It already correctly caught `1.73x` inside a Table 9 context in testing and blocked it — but if the papers reference tables/figures another way (e.g. `\cref`, `Tab. \ref{...}`), extend those two patterns.
- **Numeric extraction was buggy on first pass and has been fixed and re-tested:** the original regex missed `"1.73x"` (a trailing letter broke the word-boundary check) — it now explicitly handles `%`, `x`/×, and `s`/`sec` suffixes. Re-verify against real `_patched.tex` files before trusting it broadly, since paper prose has more unit variety than the smoke test covered.
- **False positives are cheap, false negatives are not.** The pipeline is deliberately conservative — e.g. it also flagged the bare `9` in "Table 9" as a separate (blocked) claim, which is harmless noise, not a wrong auto-fix.

7 Still open

- Real `paper_targets.json` schema (item 6 in Section 5) — blocks Tier 1's section→results resolution from working correctly until fixed.
- Whether `audit_registry.json`'s `resolved`/`open` items (2, 3, 6, 7, 8, 9, 11) should also get Tier-1 numeric checks run against them now, or be left for manual/Tier-2 handling since several are pure text fixes with no numeric component.